

Pipeline MLOps Multimodal Rakuten

Classification e-commerce texte + image — de la donnée brute à la production

1. Contexte & problématique

Rakuten, fondée au Japon en 1997 et pionnière du concept de marketplace, est aujourd'hui l'une des plus grandes plateformes e-commerce mondiales avec plus d'un milliard d'utilisateurs. Son catalogue produit, alimenté par des centaines de milliers de marchands, pose un défi majeur : extraire automatiquement les attributs des produits (catégorie, couleur, etc.) à partir de données hétérogènes et bruitées.

Sur les marketplaces de cette taille, les approches manuelles ou par règles ne passent pas à l'échelle. Pourtant, une catégorisation fiable est essentielle pour la recherche, les recommandations et la découverte produit. Le défi est triple :

- la nature multimodale des données (titre, description, image), souvent complémentaires mais parfois contradictoires ;
- le bruit intrinsèque des libellés et des images fournis par les marchands ;
- le déséquilibre des classes et la dérive (drift) progressive du catalogue dans le temps.

Au-delà du modèle lui-même, l'enjeu industriel est d'opérer ce système de manière fiable : le réentraîner, le surveiller, détecter ses dérives, et pouvoir le mettre à jour sans interruption de service. C'est précisément la promesse du MLOps.

2. Objectifs du projet

Ce projet construit une plateforme MLOps end-to-end pour la classification multimodale de produits Rakuten en 27 catégories, en se basant sur le titre, la description et l'image fournie par le marchand. Les objectifs étaient :

- concevoir un modèle multimodal performant fusionnant traitement de texte (NLP) et vision par ordinateur (CNN) ;
- industrialiser ce modèle dans une architecture microservices conteneurisée et reproductible ;
- automatiser l'ensemble du cycle de vie ML : ingestion, prétraitement, entraînement, déploiement et surveillance ;
- mettre en place un monitoring de production capable de détecter automatiquement les dérives de données et de performance.

3. Solution technique

3.1 Architecture du modèle multimodal

L'architecture retenue combine deux branches d'extraction de features, réduites indépendamment puis fusionnées avant un classifieur dense :

- **Branche texte** — nettoyage et tokenisation via spaCy (suppression des stop-words FR/EN), vectorisation par CountVectorizer (5 000 features), puis réduction via IncrementalPCA à 1 024 dimensions.
- **Branche image** — extraction de features via ResNet50 préentraîné sur ImageNet (2 048-d, global average pooling, sans top), puis IncrementalPCA à 300 dimensions.
- **Fusion et classification** — concaténation (1 324-d) → couche dense de 512 neurones (ReLU) → Dropout(0.5) → couche softmax à 27 sorties.

Le choix de l'IncrementalPCA est volontaire : il permet d'entraîner la réduction de dimension par batchs sur de très gros volumes (~250 K produits) sans saturer la mémoire, contrainte clé pour passage à l'échelle.

3.2 Architecture microservices

La plateforme est construite autour de microservices FastAPI conteneurisés, orchestrés par Docker Compose et coordonnés par Apache Airflow :

Service	Port	Rôle
gate-api	5000	Authentification JWT, RBAC, validation de tokens
train-api	8000	Entraînement asynchrone (background jobs), logging MLflow, suivi du statut
predict-api	8001	Inférence multimodale (texte / image / fusion), métriques de drift en temps réel
airflow	8080	Orchestration des DAGs : déclenchement → polling → évaluation
mlflow	5001	Tracking centralisé des expériences, registre de modèles (Staging → Production)
streamlit	8501	Interface de démonstration interactive (test du modèle, statut des services)
prometheus / grafana	9090 / 3000	Collecte de métriques + dashboard de drift à 22 panneaux + alerting

3.3 Pipeline MLOps de bout en bout

Le cycle de vie ML est entièrement automatisé et reproductible :

- **Versioning des données** — DVC suit les datasets et les artefacts (vectorizer, PCA, modèle Keras), avec stockage distant sur DagsHub (S3-compatible).
- **Pipeline DVC reproductible** — 5 étapes déclarées (preprocess_text, preprocess_image, reduce_features, train_model, predict_model) avec dépendances et sorties explicites ; rejouable en une commande (dvc repro).
- **Orchestration Airflow** — DAG asynchrone qui déclenche un POST /train, puis poll /train/status/{job_id} toutes les 60 s (timeout 4 h, jusqu'à 10 retries) — pattern robuste évitant les timeouts HTTP sur les longs entraînements.
- **Tracking & registre MLflow** — chaque run logge paramètres, métriques par epoch et artefacts ; les modèles transitent par les stages Staging → Production. Le predict-api charge automatiquement la dernière version Production avec fallback disque si le registre est indisponible.
- **CI/CD** — GitHub Actions exécute le pipeline DVC et la suite de tests (96 tests, 9 skipped) à chaque push.

3.4 Monitoring & détection de drift

La détection de dérive est un pilier du projet. Cinq familles de métriques sont collectées par Prometheus depuis chaque endpoint d'inférence et visualisées dans un dashboard Grafana de 22 panneaux :

Métrique surveillée	Type	Seuil d'alerte
Confiance moyenne (max softmax)	Drift de prédiction	< 0.5 sur 5 min
Entropie de Shannon	Incertitude du modèle	> 2.5
Distribution des classes prédites	Drift de distribution	> 50 % sur une classe
Précision validation (post-train)	Performance du modèle	< 0.60
Latence p95 d'inférence	Performance opérationnelle	> 2 s

Cette télémétrie permet d'identifier proactivement quand le modèle commence à se dégrader (drift covariate ou concept drift), avant que la dégradation ne soit visible côté utilisateur, et de déclencher un réentraînement de manière éclairée.

4. Stack technique complète

Domaine	Technologies & outils
Modélisation	TensorFlow / Keras, ResNet50 (ImageNet), scikit-learn, IncrementalPCA, spaCy
APIs & services	FastAPI, Uvicorn, JWT, Pydantic — architecture microservices (gate / train / predict)
Orchestration	Apache Airflow (DAG asynchrone, polling, retries), Streamlit (UI démo)
MLOps / tracking	MLflow + DagsHub (registre de modèles, S3 artefacts), DVC (versioning données)
CI/CD	GitHub Actions (pipeline DVC + tests), pytest (96 tests, unit + intégration)
Conteneurisation	Docker, Docker Compose (10+ services orchestrés), PostgreSQL, MinIO
Monitoring	Prometheus, Grafana (dashboard 22 panneaux), Alertmanager — détection de drift

5. Résultats & livrables

Le projet livre une plateforme MLOps complète, fonctionnelle et reproductible :

- Architecture microservices à 10+ conteneurs orchestrés par un unique docker-compose up ;
- Suite de tests automatisée : 96 tests unitaires et d'intégration validant les APIs, le pipeline ML, l'authentification JWT et le cycle de vie des jobs asynchrones ;
- Pipeline d'entraînement asynchrone capable de traiter ~85 000 produits par lot sans saturer la mémoire (IncrementalPCA + batching) ;
- Dashboard de drift à 22 panneaux opérationnel et 5 règles d'alerting Prometheus / Alertmanager configurées ;
- Versioning complet : code (Git), données et artefacts (DVC + DagsHub), modèles (MLflow Model Registry) ;
- Démonstrateur Streamlit permettant de tester en direct l'inférence texte, image ou multimodale via l'API protégée par JWT.

6. En résumé

Ce projet illustre un cas concret de mise en production d'un modèle de Machine Learning multimodal dans une logique industrielle MLOps. Au-delà de la performance brute du classifieur, il traite l'ensemble des problématiques opérationnelles auxquelles une équipe Data / ML est confrontée : reproductibilité des entraînements, déploiement sans rupture, observabilité, détection de dérive et automatisation du cycle de vie.

L'architecture choisie — microservices FastAPI, orchestration Airflow, tracking MLflow centralisé sur DagsHub, monitoring Prometheus / Grafana — est directement transposable à un environnement de production réel et reflète les standards actuels de l'industrie ML.

Liens du projet

Code source : github.com/zz75da/rakuten_z

Données & expériences MLflow : dagshub.com/zz75da/rakuten_z